

Foreword

It seems like Python is very relevant for coding in the modern day, but there are not many resources available if you are a student who wants to pick up the language at the level of a sophomore CS student. People in this position have a strong grasp of the fundamental knowledge necessary to understand the basics (and therefore, do not need to learn at the level of a beginner), but may not benefit from the excessive jargon and features most technical resources explain. With that being said, I have decided to create this learning guide to gloss over the elementary concepts in Python that should already be familiar to competent sophomore CS students, but explain notable differences in syntax and keep advanced topics palatable to our current level of knowledge in C++. Hopefully by the end, you will be able to write in Python at a similar level as a sophomore or junior who learned C++ in class.

Table of Contents:

Semester 1 Part 1

- Overview of Python
- Syntax
- Variables and Primitive Data Types
- Input, Output, and Checking for Input Failure
- Conditional Statements and Loops (if, while, for)

Semester 1 Part 2

- Logical Short Circuiting and Other Quirks
- Common Libraries (Equivalents to `iostream`, `cmath`, `iomanip`, `fstream`)
- File Reading and File Input/Output
- Arrays and Indexing
- Functions

Semester 2 Part 1

- **Classes**

- Member access specifiers
- Constructors
- Inheritance (single and multiple)
- 3 principles of object oriented design
- Static member
- Automatic member

- **Pointers**

- Shallow vs deep copy

Semester 2 Part 2

- **Exception Handling**

- Try catch
- How exceptions are caught and thrown

Overview of Python

What is Python?

Python is a high-level, interpreted programming language known for its readability and versatility. It was created by Guido van Rossum and released in 1991. It has gone through many versions since then, and Python 3 is the current standard.

Key Characteristics

Compiled vs Interpreted

Unlike Python, C++ is a compiled language. Compiled programs are translated from source code into a standalone binary file (like an .exe) before you run it. They have better performance and efficiency and errors are often caught before the program ever runs. However, you must re-compile the code every time you make a change. Additionally, the resulting file is usually specific to one operating system (e.g., Windows vs. Linux). This can potentially make development slower, as compiling large programs can take a lot of time.

In languages like Python, a program called an interpreter executes your program and runs the entire time the program is active. This means the program starts immediately without a separate compiling step, but the extra work of translating while running makes it slower to run compared to compiled code.

The interpreter is portable across different operating systems, so on any system the interpreter is installed, your code can be run. Additionally, it can be easier to develop and test since you do not have to wait for your program to be compiled. However, some errors that a compiler might have caught may only appear halfway through with interpreted programs.

Simpler Syntax

Unlike C++ which uses semicolons, Python uses indentation making it easy for beginners to learn and professionals to maintain. This means you can simply press tab any time you need to indicate a for loop or similar blocks of code instead of using curly braces like C++. Do note that technically the standard is to use 4 spaces to mark indentation instead of just the Tab key (to resolve potential visual clarity issues across platforms), but most modern IDEs will automatically convert Tab into 4 spaces. Just make sure you check yourself before running into any potential IndentationErrors. Additionally, you must ensure your indentation is correct or your interpreter will give you an error or a logic bug could occur.

Comment Syntax

Comments in Python are denoted by the hash symbol (#). There is no long-form comment syntax, so multi-line comments commonly just use consecutive hashes. Alternatively, you can use `"""` or `'''` to create a multi-line string which can be used to mimic multi-line comments. If it is not assigned to a variable, Python will ignore it during execution, though it is technically a string literal and not a true comment.

Example:

```
# Method 1: Consecutive hash marks
```

```
# This is a multiline comment
```

```
"""
```

```
Method 2: Triple-quoted strings
```

```
This is often used for multiline comments
```

```
'''
```

Dynamic Typing

You don't need to declare variable types; the interpreter instead determines them at runtime. You can also reassign variables to other types at any time throughout the program (e.g. change a string to an integer). If you try to apply an operation to invalid types, only then does the interpreter give an error. Note that if you want to overload operators for more types, you will have to use special Methods.

Large Standard Library

The standard library for Python contains most common modules needed, so you don't need a large amount of `#include(s)`. One common complaint of C++ is that the `#include` system is convoluted and archaic, and Python makes this aspect easier to deal with.

Real World Applications

Coding is common in many engineering and computer science related fields, but some fields benefit more from fast to develop, easy to use languages like Python versus those that prioritize performance and speed like C++. Another approach that makes Python easier is "wrapping", which means you can make your engine in C++ to get the performance where it's needed but write your code in Python to make it easier to use. This makes it easier for those not directly in computer science fields to contribute and read code.

Common fields that use Python include:

Data Science, AI, and Machine Learning (with wrappers)

Scientific Simulations and Computing (with wrappers)

Web Development Backend

Automation and Simple Scripts (e.g. web scrapers)

Variables and Primitive Data Types

There are only 4 primitive data types in Python, as there is no “char” type. Char is instead handled as a string with length 1. Additionally, there is no type declaration and no pointer data type. This means you simply declare variables with their name, assignment, and their value. Additionally, you can reassign a variable to a different type at any time.

Note that unlike C++, integers automatically grow in size to handle numbers as large as your computer's memory allows, and all floats are handled as doubles.

Examples:

```
name = "Alice"    # str
age = 25          # int
height = 5.7      # float
is_student = True # bool
```

• Input, Output, and Checking for Input Failure

C++ tries to read data directly into a specific data type from the stream. Python simply grabs whatever the user typed as a string, and it is up to you to convert it.

Output (print)

To display information to the user, Python uses the built-in `print()` function. You can pass strings, variables, or a mix of both into it.

```
name = "Alice"    # str
print("Hello, world!") #prints the given string
print(f"Welcome, {name}!") # using f string
print("Hello, " + name + ". You are " + str(age) + " years old.") # using string concatenation
```

Here you can use what is called an f string, (formatted string literal) which allows you to put variables in line with the strings without using concatenation. The syntax is an f before your string, and curly braces around your variables. You can also call methods, do arithmetic, and format displayed information using some tricks within those curly braces, but you can investigate that more yourself.

Input (input)

To get information from the user, Python uses the `input()` function. It pauses the program, waits for the user to type something and press Enter, and then returns whatever they typed as a string.

```
user_age = input("Please enter your age: ")  
# Note: user_age is a string (e.g., "25"), even if they typed numbers.
```

The `input()` function always reads the user's input as a string. There is no concept of a "failed stream" like in C++. Input failure only happens when you attempt to cast that string into another type, like an integer. If you try `int("hello")`, Python throws a `ValueError` exception. To deal with this, you need to use exception handling and while loops to get valid data.

```
while True:    #infinite loop until break statement  
    user_input = input("Enter an integer: ")  
    Try:  
        # Attempt to convert the string to an integer  
        my_int = int(user_input)  
        print(f"Success! You entered {my_int}")  
        break # Exit the loop if successful  
    except ValueError:  
        # This catches the specific error of a failed conversion  
        print("Invalid input. That is not an integer.")
```

- Conditional Statements and Loops (if, while, for)

Python strips away punctuation found in C++, replacing parentheses and curly braces with colons and strict whitespace indentation. Parentheses are optional (and generally discouraged unless grouping complex logic). Curly braces are replaced by a colon : at the end of the statement, and the block of code must be indented. Python uses the portmanteau elif instead of else if.

```
x = 10
if x > 10:
    print("Greater")
elif x == 10:
    print("Equal")
else:
    print("Smaller")
```

While Loops

The logic is identical in both languages, just make sure the whitespace syntax is correct. Note that C++ features a do...while loop (which guarantees the code runs at least once before checking the condition). Python does not have a do...while loop. You typically simulate it using a while True: loop with an if condition: break at the end.

```
while countdown > 0:
    print(countdown)
    countdown -= 1 # Decrease countdown by 1 each time
```

Note: there are no increment nor decrement operators like in C++. Instead, you use += or -= and add or subtract your variable by the amount on the right.

For Loops

Python does not have a traditional C-style for loop. Python loops iterate directly over items in a collection (like a list or a string). If you just want to loop a specific number of times (like a traditional C++ loop), you use Python's built-in `range()` function, which generates a sequence of numbers.

```
# Equivalent to C++: for(int i = 0; i < 10; i++)
```

```
for i in range(10):
```

```
    print(i)
```

```
# Equivalent to C++: for(string name : names)
```

```
names = ["Alice", "Bob", "Charlie"]
```

```
for name in names:
```

```
    print(name)
```

As you can see, you first declare a new variable that holds each individual entry in a collection, then a variable that holds a given collection or range. Due to the way this kind of for loop works, you cannot easily make changes to “i” mid loop.

Additionally, modifying a collection while iterating over it is more dangerous, as it makes changes to the actual collection. This can cause bugs when you add or remove items from a list mid loop. To remedy this, you can iterate on a copy of the collection rather than the collection itself.

Semester 1 Part 2

• Logical Short Circuiting and Other Quirks

Both C++ and Python utilize short-circuit evaluation for logical operations (AND / OR). This means the evaluation of a logical expression stops as soon as the final outcome is determined. While the mechanism is identical, what these operators actually return is completely different between the two languages.

C++ returns a Boolean (true or false): The logical operators `&&` and `||` always evaluate strictly to a bool.

Python returns the Object itself: Python's "and" and "or" operators return the value of one of the operands, not necessarily a boolean.

This is because python does implicit boolean conversion. Any variable can be treated as a boolean, which can have "truthy" or "falsy" values.

In C++, numbers equal to 0 or nullptr evaluate to false, and everything else is true. In Python, an object is considered Falsy if it is:

A number equal to zero (0, 0.0)

None (Python's equivalent to nullptr)

Any empty collection (an empty string "", empty list [], empty dictionary {}, etc.)

All other values of all variable types are considered true, or "truthy".

Chained Comparisons

Another quirk of Python is chained comparisons. Basically, unlike C++ where each comparison needs to be separated and explicit, Python supports chaining comparison statements (e.g. the statement `0 < x < 10` is valid in Python, while C++ requires `(0 < x && x < 10)`).

Loop else Blocks

In Python, both for and while loops can have an optional else block attached to them. This does not exist in C++. The else block executes only if the loop finishes normally without hitting a break statement. This is useful when searching via a loop, as you can have a dedicated case for when an object or property is not found among a collection.

- Common Libraries (Equivalents to `iostream`, `cmath`, `iomanip`, `fstream`)

In Python, many of the standard libraries you have to `#include` are built in. There are some that you still have to include, which you do using “import”.

`<iostream>`, `<iomanip>`, and `<fstream>` are all built in. You simply use the corresponding syntax. However, some libraries, like `<cmath>`, still require external libraries. In the case of `cmath`, you would use “import math” at the start of your code.

- File Reading and File Input/Output

C++ requires you to instantiate a stream object, explicitly open it, check if it successfully opened, and manually close it.

Python utilizes a Context Manager (with statement) to handle files. The with block guarantees that the file is safely closed the exact moment the code block ends, even if an error occurs mid-read. There is technically still a `.close()`, but most of the time you will just rely on the with statement to do it for you.

```
# The 'with' statement automatically closes the file when done
with open("example.txt", "w") as out_file:
    out_file.write("Hello World")
```

• Arrays and Indexing

Python uses lists in the place of arrays and vectors. Unlike C++, there is no static size array, instead arrays are dynamic, more like vectors. However, unlike vectors, you can put any type of data in a single list.

A C++ array holds the actual raw data values right next to each other. A Python list holds a contiguous array of pointers that point to objects scattered elsewhere in memory, similar to a linked list (hence the name list).

Indexing and Bounds Checking

If you have an array of size 5 and try to access `arr[10]`, C++ has no bounds checking by default. It will happily look at whatever garbage data is sitting at that memory address. Python, however, strictly checks boundaries. If you try to access an index that doesn't exist, Python immediately halts execution and throws an `IndexError`.

You can also index negative numbers to index backward from the end of the list.

List Slicing

In C++, if you want to copy elements 2 through 4 of an array into a new array, you have to write a for loop or use `std::copy`. In Python, you can use built-in syntax called slicing, using the format `list[start:stop:step]`. Note that the stop index is exclusive (it goes up to, but does not include, that index).

```
numbers = [0, 10, 20, 30, 40, 50, 60]

# Grab elements from index 1 up to (but not including) index 4
sub_list = numbers[1:4]
print(sub_list) # Output: [10, 20, 30]
#Note: if one colon is omitted, the format defaults to [start : stop]

# Slice with a step (grab every second item)
print(numbers[0:6:2]) # Output: [0, 20, 40]

# Reverse an entire list instantly using a negative step
print(numbers[::-1]) # Output: [60, 50, 40, 30, 20, 10, 0]
#Note: if the arguments for start or stop are omitted, it will start
#at the beginning and end at the end respectively
```

If you are doing heavy mathematical operations or data science where you need the more memory-efficient fixed-type arrays, you don't use lists. Instead, you import a library called NumPy, which provides true C-speed arrays (`numpy.array`).

• Functions

Functions in Python are defined using the `def` keyword. Because Python is dynamically typed, you do not declare parameter types or return types.

```
# Python function definition
def add(a, b):
    return a + b
```

This particular function naturally acts like a C++ template function. It will accept anything that supports the `+` operator. If you pass two integers, it adds them. If you pass two strings, it concatenates them. This is called Duck Typing ("If it walks like a duck and quacks like a duck, it's a duck").

Pass by Value/Pass by Reference

C++ gives you explicit control over memory with pass-by-value (`void func(int x)`) and pass-by-reference (`void func(int &x)`).

Python uses a mechanism called Pass-by-Object-Reference (or pass-by-assignment). You don't get to choose between value or reference; Python's behavior depends entirely on whether the object you are passing is mutable (changeable) or immutable (unchangeable).

Immutable objects (primitive data, like ints, floats, and strings) act like pass-by-value. If you modify them inside the function, Python leaves the original variable outside the function completely untouched.

Mutable objects (data collections like lists, dictionaries, and sets) act like pass-by-reference. Because a list can be modified in place, any changes you make to its contents inside the function will affect the original list outside the function.

Function Overloading vs. Default Parameters

Python does not support function overloading. If you write two functions with the exact same name, the second one will simply overwrite the first one. However, you can use Default Parameters and Keyword Arguments to achieve a similar effect.

```
# A single function handling multiple use-cases via default values
def greet(name, message="Hello", punctuation="!"):
    print(f'{message}, {name}{punctuation}')

# You can call it matching the positional order:
greet("Alice")           #Hello, Alice! (Default message and punctuation + {name})
greet("Bob", "Good morning") #Good morning, Bob! (Default punctuation + {message} + {name})

# Or you can explicitly mention Keyword Arguments to jump out of order:
greet("Charlie", punctuation="!!!") # Hello, Charlie!!!
greet(punctuation=".", name="Dana") # Hello, Dana.
```

Returning Multiple Values

In C++, a function can only return one thing natively. If you need to return multiple values, you have to use pass-by-reference, return a pair or tuple, or build a custom struct.

Python lets you natively return multiple values separated by commas. Under the hood, Python bundles them into a tuple, which you can instantly unpack.

```
def get_min_max(numbers):
    return min(numbers), max(numbers) # Returns a tuple: (min, max)

# Unpacking the returned values into separate variables instantly
lowest, highest = get_min_max([4, 1, 9, 2, 8])
print(f'Low: {lowest}, High: {highest}')
```

Semester 2 Part 1

Classes

In C++, a class is a compile-time blueprint. Once the program compiles, the structure of the class is locked in stone. Any changes to the original class need to be explicitly mentioned in the class definition.

In Python, a class is actually an object itself, evaluated at runtime.

Any function within the class can be redefined with a new function using simple assignment (=), new functions can be created at any time, and individual instances of the class can be modified to have unique properties. This also applies to variables and arrays applied to the class. This practice is called monkey patching, and while it is useful for testing experimental features, it does make your code harder to read. Typically, any final changes to a class you make should be done via changing the class definition or defining inheritance.

The constructor is always defined using `__init__(self, [class variable], ...)`. Instead of defining the variables to the class itself, you define them within the constructor. There is no constructor overloading, so if you want to achieve a similar effect, you can use the default arguments of the constructor or class methods to achieve a similar effect.

There also isn't an implicit this pointer for a specific class instance. When constructing your class, you must always first define an implicit pointer, with the typical naming convention being "self". After that, you can define any number of variables to the class that you want.

```
class Car:
    # No need to define 'model' variable yet

    # Constructor (Initializer)
    def __init__(self, model):
        self.model = model # Created dynamically on the fly

    def describe(self):
        print(f"This is a {self.model}")
```

One thing you may also notice is that there are no member access specifiers (public, protected private). Python just expects developers to not mess with important classes. You can put a single underscore before variable names as a signal to not act on a variable, or use a double underscore to make it a little more difficult to access (but not impossible).

Also, there are no destructors, copy constructors, etc. Memory management for a class is handled by Python automatically.

Static Members

In C++, a static member belongs to the class as a whole, not instances, and must be declared with the static keyword.

In Python, variables declared directly inside the class body but outside any methods are naturally static (called Class Variables). They are shared by every instance of that class.

```
class Item:
    count = 0 # STATIC MEMBER / Class variable

    def __init__(self):
        Item.count += 1 # Increments the shared class counter
```

Automatic Members (Instance Variables)

In Python, Instance Variables are created dynamically on the fly, typically inside the `__init__` constructor, by prefixing them with `self.[attribute]`.

```
class Item:
    def __init__(self, name):
        self.name = name # AUTOMATIC/INSTANCE MEMBER unique to this specific object
```

An interesting (but mostly useless) quirk of Python is that because objects are dynamic dictionaries, you can actually attach a brand-new instance variable to an individual object retroactively:

```
toy = Item("Rubik's Cube")
toy.color = "Mixed" # This unique instance now has a 'color' attribute!
```

3 Principles of Object Oriented Design

If you recall, the 3 principles of Object-Oriented Design are encapsulation, inheritance, and polymorphism. As a recap...

Encapsulation is the practice of bundling the data and the functions that operate on that data into a single class. It involves hiding the internal state and implementation details of an object and only exposing a controlled, public interface.

Inheritance is a mechanism that allows a derived class (or child class) to inherit members and functions from a base class (or parent class). The derived class reuses the code of the base class and can extend it with new features or override existing ones.

Polymorphism is the ability of objects of different classes to respond to the same function call in different, class-specific ways, and for objects of different types to be treated as objects of a common base type. It allows a single parent class to be used for multiple distinct child classes.

The means by which these principles are achieved differ, but they are all possible within Python.

Encapsulation

As mentioned earlier, there are no member access specifiers in Python. However, a controlled public interface for variables and functions can still be achieved through using decorators.

Decorators

To understand how encapsulation works in Python, we first need to understand what a decorator is. A decorator is a function that takes another function as an argument, extends its behavior without explicitly modifying it, and returns a new function.

In terms of functions, when a function with a defined decorator is used, before any code from that function is run, Python first executes the function associated with the decorator. Once that function is run, the original function runs as expected. This is useful for encapsulation, as it allows you to isolate main function logic from secondary tasks.

The @property Decorator

Using Object-Oriented Programming in C++, you achieve data encapsulation by making class variables private and writing explicit getter and setter functions to control access. Python doesn't have true "private" variables, but it achieves data encapsulation using the built-in @property decorator. This allows you to protect an internal attribute while exposing what looks like a normal, public variable to the outside world.

The Getter (@property)

This is the main decorator. It turns a method into a "read-only" attribute. It is responsible for returning the hidden internal value (which we usually denote with a single leading underscore, like `_celsius`).

```
class Temperature:
    def __init__(self, celsius):
        # We store the actual data in a "private" internal variable
        self._celsius = celsius

    @property
    def celsius(self):
        # The Getter: runs when someone tries to READ the value
        return self._celsius
```

Now, if you type `print(temp.celsius)`, Python secretly runs this method and returns the value. At this stage, you cannot change the value without the dedicated setter function. If you try `temp.celsius = 100`, Python will throw an `AttributeError`.

The Setter (@[name].setter)

To allow users to change the value securely, we define a setter. The name of the decorator must be the name of your property, followed by `.setter`. This is where we put our validation logic.

```
@celsius.setter
def celsius(self, value):
    # The Setter: runs when someone tries to CHANGE the value
    if value < -273.15:
        raise ValueError("Temperature cannot go below absolute zero!")
    self._celsius = value
```

Now, when you type `temp.celsius = 25`, Python takes the 25, passes it into the setter method as the value argument, and runs your check before updating `_celsius`.

3. The Deleter (@[name].deleter)

This is less common, but if you want to run specific cleanup code when someone tries to delete the attribute using the `del` keyword, you can use the deleter decorator. Since Python has automatic memory management, you will not need it most of the time.

Inheritance (single and multiple)

Instead of using a colon (class Car : public Vehicle), Python passes the parent class(es) inside parentheses during the class definition (class Car(Vehicle):).

Single Inheritance

You can achieve single inheritance via the super() function called within the child class constructor.

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle): # Inherits from Vehicle
    def __init__(self, brand, wheels):
        super().__init__(brand) # Calls Vehicle's constructor
        self.wheels = wheels
```

Multiple Inheritance

When programming in C++, you have to worry about the diamond inheritance problem when two parent functions try to call the same grandparent. In Python, an algorithm called MRO (Method Resolution Order) is utilized to address this. It checks classes from left to right in the parentheses to determine which parent method to run.

```
class Flyer:
    def move(self): print("Flying")

class Swimmer:
    def move(self): print("Swimming")

class Duck(Flyer, Swimmer): # Inheriting from both
    pass

d = Duck()
d.move() # Output: "Flying" always output because Flyer was listed first in the MRO
```

Note: pass is used here after the Duck(...) class because all functions/classes need at least 1 indented line. Otherwise, you will get an IndentationError even if your class doesn't actually contain anything. However, one-line blocks can be written on the same line as the colon.

Polymorphism

Polymorphism in Python is inherently built into the language through a concept known as Duck Typing, alongside effortless method overriding and operator overloading.

In C++, to pass different objects into a single function, those objects must inherit from a common base class that defines a virtual method. In Python, inheritance is not required. As long as the object passed into a function has the method being called, Python will execute it happily.

```
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

class Robot:
    # Does not inherit from any animal class!
    def speak(self):
        return "Beep boop."

# A single polymorphic function
def make_it_speak(entity):
    # Python doesn't care what type 'entity' is,
    # it only cares if it has a 'speak()' method.
    print(entity.speak())

make_it_speak(Dog()) # Output: Woof!
make_it_speak(Robot()) # Output: Beep boop.
```

Overriding

When you do use inheritance, Python allows child classes to override parent methods by simply defining a method with the exact same name.

There is no need to mark the parent method as virtual, and there is no need to use an override keyword in the child class. Python always looks at the lowest level of the object (the child) first when resolving a method call. So if you have a Dog class that inherits from Animal, and both have a speak function, the Dog speak function will always execute.

Operator Overloading

If you recall, operator overloading allows you to define how common operators function with custom classes. Python achieves this through "dunder" (double underscore) methods. Every built-in operator in Python maps to a specific dunder method. If you want to define how two custom objects interact when added together, you simply override the `__add__` method within your class.

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    # Overloads the '+' operator
    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

    # Overloads the string representation for printing
    def __str__(self):
        return f"({self.x}, {self.y})"

v1 = Vector2D(2, 3)
v2 = Vector2D(4, 1)

# Polymorphism in action: Python translates '+' to v1.__add__(v2)
v3 = v1 + v2
print(v3) # Output: (6, 4)
```

Pointers

Because Python manages memory automatically via a Garbage Collector and uses dynamic typing, almost all of the manual memory juggling from C++ simply doesn't exist. This means that we can completely skip C++ concepts like:

- Manual allocation and deallocation (new and delete)
- Pointer syntax (* and &)
- Memory leaks and dangling pointers
- Handling pointers within classes (e.g., custom copy constructors, assignment operators, and destructors)

Of course, it is still possible to do memory management in rare circumstances with special libraries, but there is not really a point in explaining that in reference material of this level. There is still one concept related to pointers that must be addressed, however.

Shallow and Deep Copies

If you make a copy of an array using assignment (=), that array will simply be an alias, or a second variable label pointing to the same memory address.

To make a distinct copy, you need to use the dedicated copy module.

Copy Module

This is a library you can use by importing “copy”. It allows you to make copies of arrays that preserve the original array. There are two main functions that are relevant for our purposes.

```
copy.copy([variable name])
```

This allows you to make copies of 1D arrays or dictionaries.

```
copy.deepcopy([variable name])
```

This allows you to make copies of more complex structures, like classes or higher dimension arrays. If you try to use `copy.copy()` on these types of data instead, it will only copy the immediate dimension of the array, but the inner arrays or data will still point to the original addresses (which is obviously problematic if you are trying to make a deep copy).